Chapter 3
Memory Management

- Fetch Strategies
- Page Replacement
  - -Optimal -FIFO -LRU -NRU
    -Second-chance
- Design Issues for paging systems
- Implementation Issues
- Segmentation
- Summary

Paging Policies

- Fetch Strategies
  - When should a page be brought into primary (main) memory from secondary (disk) storage.
- Replacement Strategies
  - Which page now in primary storage is to be removed from primary storage when some other page or segment is to be brought in and there is not enough room.
- Clean Strategies
  - When a page in memory is brought out?

Demand Fetching

- Algorithm Never bring a page into primary memory until it's needed.
  1. Page fault occurs.
  2. Check if a valid virtual memory address. Kill job if not.
  3. If valid reference, check if it's cached in memory already (perhaps for some other process.) If so, skip to 7.
  4. Find a free page frame.
  5. Map address into disk block and fetch disk block into page frame. Suspend user process.
  6. When disk read finished, add vm mapping for page frame.
  7. If necessary, restart process.

Prepaging

- Algorithm bring a page into primary in advance.
- When page-fault happens, the page needed and adjacent pages will be brought into memory.
  - Advantage: improve the I/O efficiency when fetch page.
  - Disadvantage: based on prediction, if the fetched page is rare referenced, low efficiency.
  - Used when loading page.

Page Replacement

1. Find location of page on disk
2. Find a free page frame
   ① If free page frame use it
   ② Otherwise, **select a page frame using the page replacement algorithm**
   ③ Write the selected page to the disk if necessary and update any necessary tables
3. Read the requested page from the disk.
4. Restart the user process.

Page Replacement Strategies

- **The Principle of Optimality**
  - Replace the page that will not be used again the farthest time in the future.
- **Random page replacement**
  - Choose a page randomly
- **FIFO - First in First Out**
  - Replace the page that has been in primary memory the longest
- **NRU - Not Recently Used**
  - An approximation to LRU.
- **LRU - Least Recently Used**
  - Replace the page that has not been used for the longest time
- **NFU - Not Frequently Used**
  - Replace the page that is used least often
- **Working Set**
  - Keep in memory those pages that the process is actively using.

Principal of Optimality

- Description:
  - Assume that each page can be labeled with the number of instructions that will be executed before that page is first references, i.e., we would know the future reference string for a program.
  - Then the optimal page algorithm would choose the page with the highest label to be removed from the memory.
- Impractical because it needs future references
- If future references are known
  - should not use demand fetching
  - should use pre paging to allow paging to be overlapped with computation.
- This algorithm provides a basis for comparison with other schemes.

FIFO Page Replacement Algorithm

- Maintain a linked list of all pages
  - in order they came into memory with the oldest page at the front of the list.
- Page at beginning of list replaced
- Advantage: easy to implement
- Disadvantage
  - page in memory the longest (perhaps often used) may be evicted

Optimal Example

12 references,
7 faults

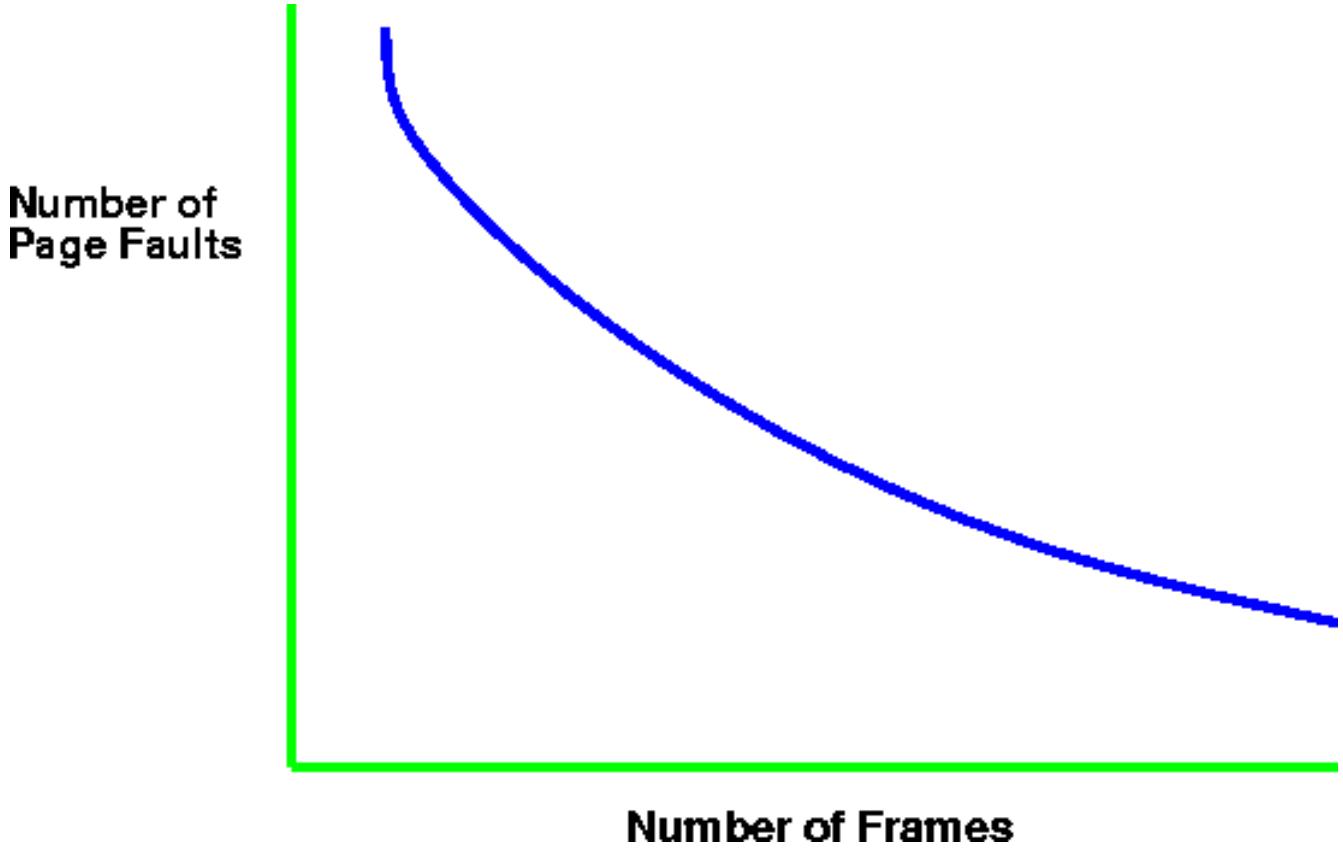
Page Refs	3 Page Frames			
	Fault?	Page Contents		
A	yes	A		
B	yes	B	A	
C	yes	C	B	A
D	yes	D	B	A
A	no	D	B	A
B	no	D	B	A
E	yes	E	B	A
A	no	E	B	A
B	no	E	B	A
C	yes	C	E	B
D	yes	D	C	E
E	no	D	C	E

FIFO

12 references,
9 faults

Page Refs	3 Page Frames			
	Fault?	Page Contents		
A	yes	A		
B	yes	B	A	
C	yes	C	B	A
D	yes	D	C	B
A	yes	A	D	C
B	yes	B	A	D
E	yes	E	B	A
A	no	E	B	A
B	no	E	B	A
C	yes	C	E	B
D	yes	D	C	E
E	no	D	C	E

Paging Behavior with Increasing Number of Page Frames



Belady's anomaly

- Belady discovered more page frames might not always have fewer page faults. This is called **Belady's anomaly**.
- A paging system can be characterized by three items:
  - The reference string of the executing process.
  - The page replacement algorithm.
  - The number of page frames available in memory.

Belady's Anomaly (for FIFO)

As the number of page frames increase, so does the fault rate.

12 references,

10 faults

Page Refs	4 Page Frames				
	Fault?	Page Contents			
A	yes	A			
B	yes	B	A		
C	yes	C	B	A	
D	yes	D	C	B	A
A	no	D	C	B	A
B	no	D	C	B	A
E	yes	E	D	C	B
A	yes	A	E	D	C
B	yes	B	A	E	D
C	yes	C	B	A	E
D	yes	D	C	B	A
E	yes	E	D	C	B

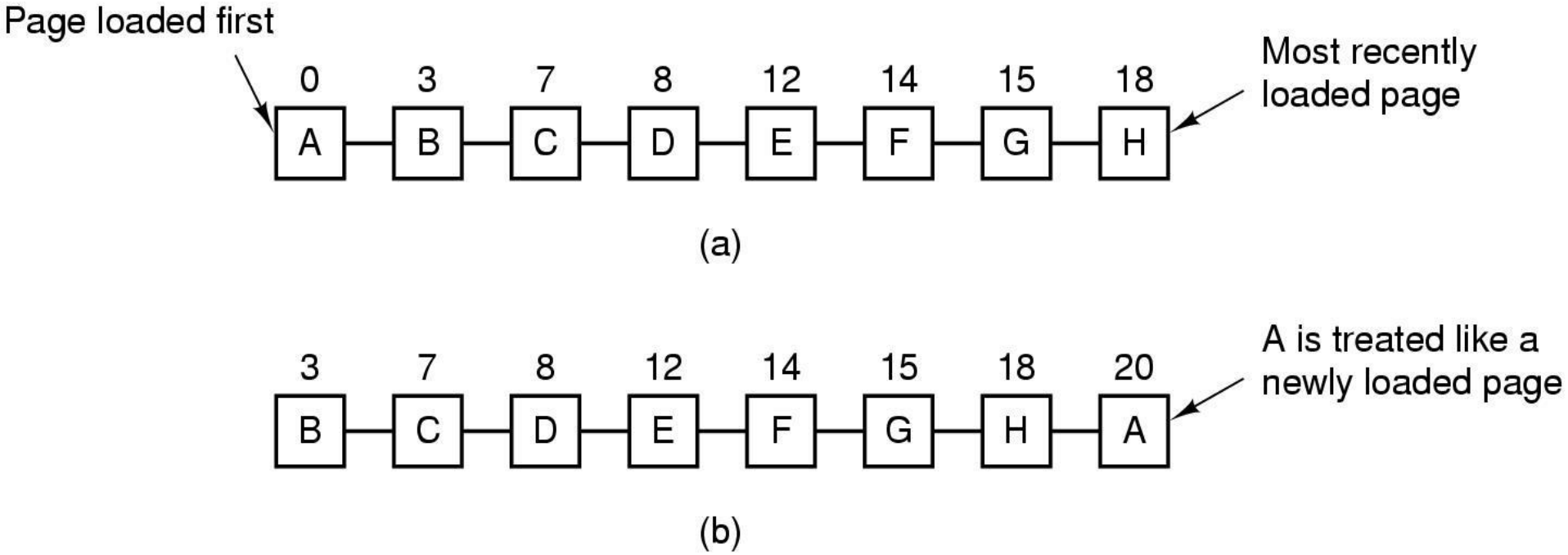
Second Chance Page Replacement Algorithm

- Inspect R bit:

  if R = 0 ➔ evict the page

  if R = 1 ➔ set R = 0 and put page at end (back) of list. The page is treated like a newly loaded page.

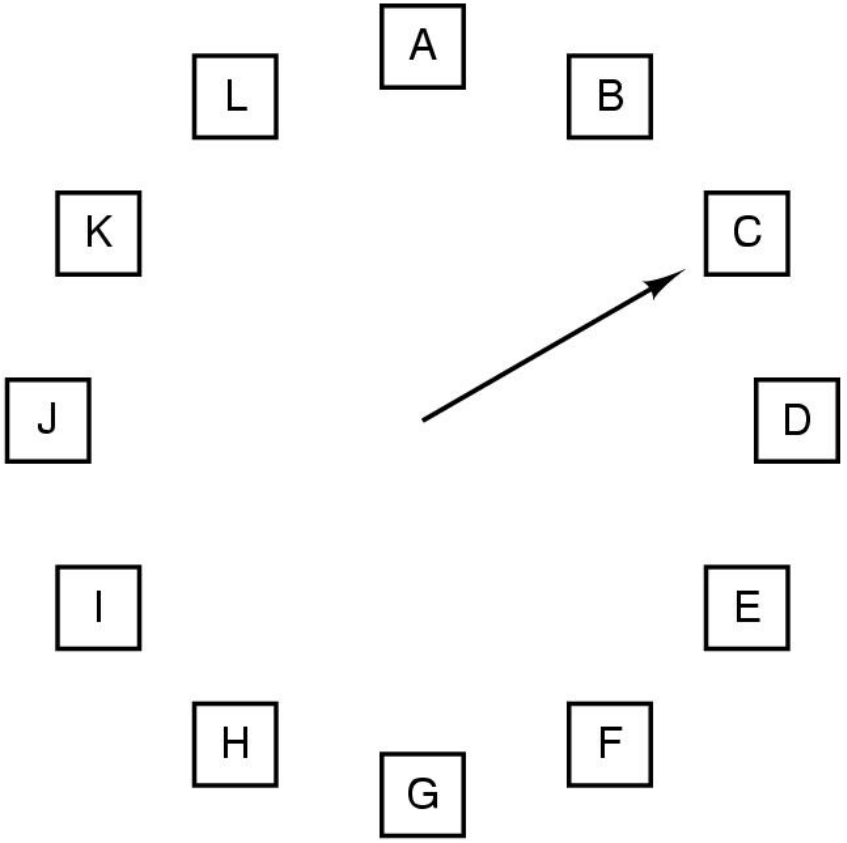
Second Chance Page Replacement Algorithm



- Operation of a second chance
  a) Pages sorted in FIFO order
  b) Page list if fault occurs at time 20, A has *R* bit set (numbers above pages are loading times)

The Clock Page Replacement Algorithm

- Clock Replacement Algorithm: a different implementation of second chance

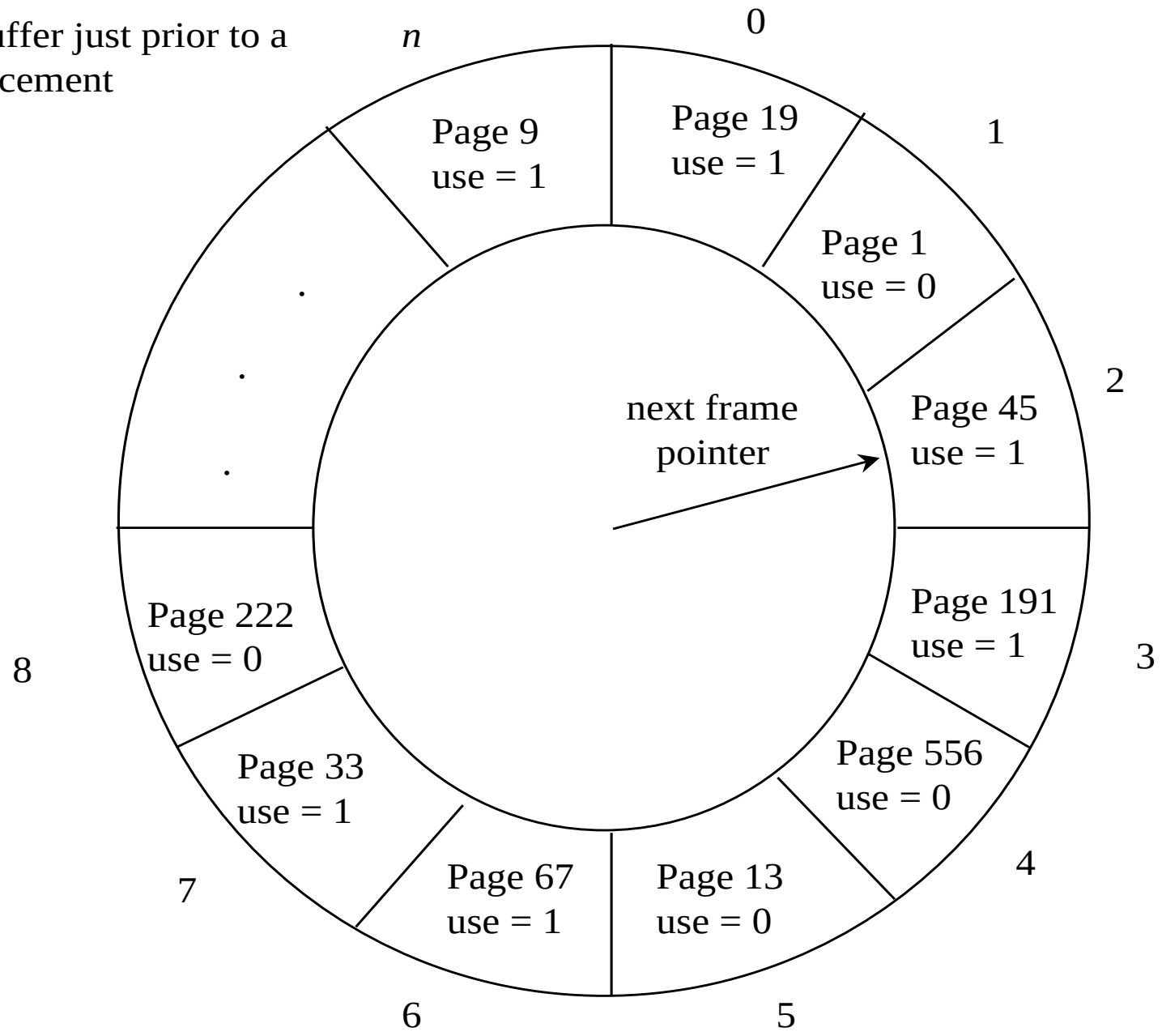


When a page fault occurs, the page the hand is pointing to is inspected. The action taken depends on the R bit:

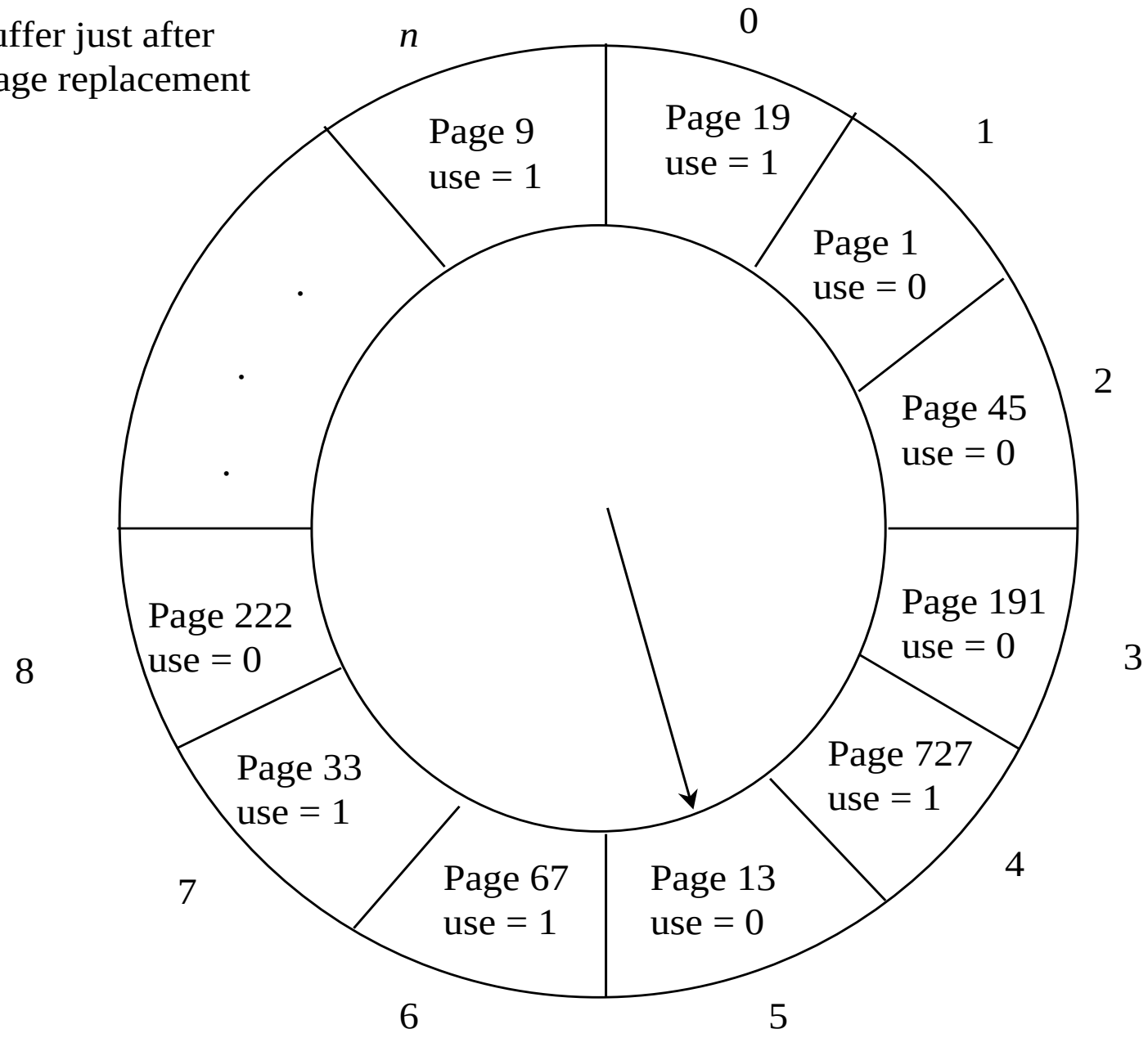
R = 0: Evict the page

R = 1: Clear R and advance hand

State of buffer just prior to a page replacement



State of buffer just after
the next page replacement



Not Recently Used Page Replacement Algorithm (NRU)

- Each page has Reference bit(R) and Modified bit(M).
  - bits are set when page is referenced (read or written recently), modified (written to)
  - when a process starts, both bits R and M are set to 0 for all pages.
  - periodically, (on each clock interval (20msec)), the R bit is cleared. (i.e. R=0).
- Pages are classified

Class 0: not referenced, not modified

Class 1: not referenced, modified

Class 2: referenced, not modified

Class 3: referenced, modified

- NRU removes page at random
  - from lowest numbered non-empty class

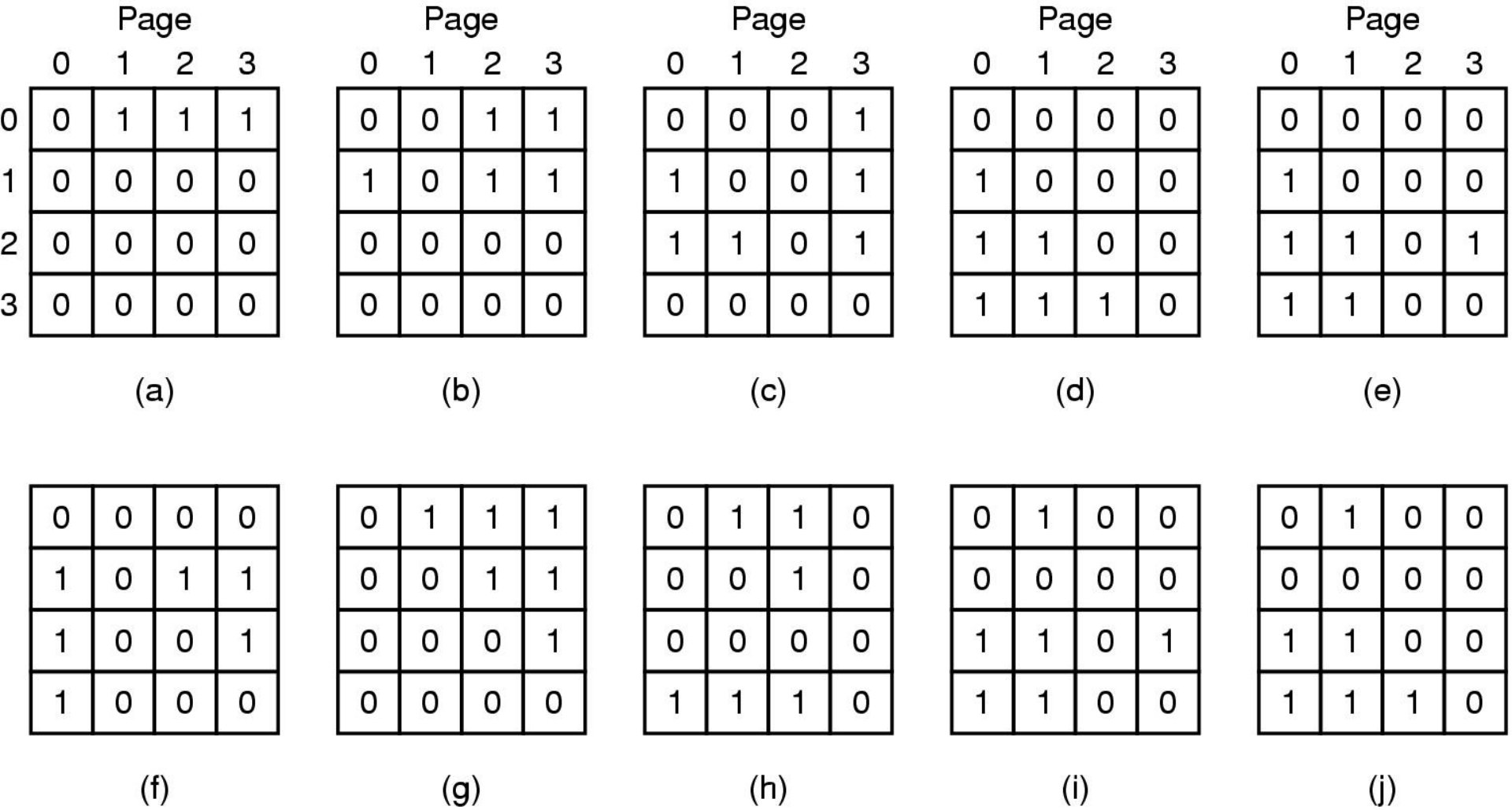
Least Recently Used (LRU)

- Assume pages used recently will used again soon
  - throw out page that has been unused for longest time
- Software Solution: Must keep a linked list of pages
  - most recently used at front, least at rear
  - update this list every memory reference ➔ Too expensive!!
- Hardware solution 1: Equip hardware with a 64 bit counter.
  - That is incrementing after each instruction.
  - After memory reference, the counter value is stored in the page table entry of the page that was just referenced.
  - choose page with lowest value counter
  - periodically zero the counter

Least Recently Used (LRU)

- Hardware solution 2:
  - Maintain a matrix of n x n bits for a machine with n page frames.
    - When page frame K is referenced:
      (i) Set row K to all 1s.
      (ii) Set column K to all 0s.
    - The row whose binary value is smallest is the LRU page.

LRU

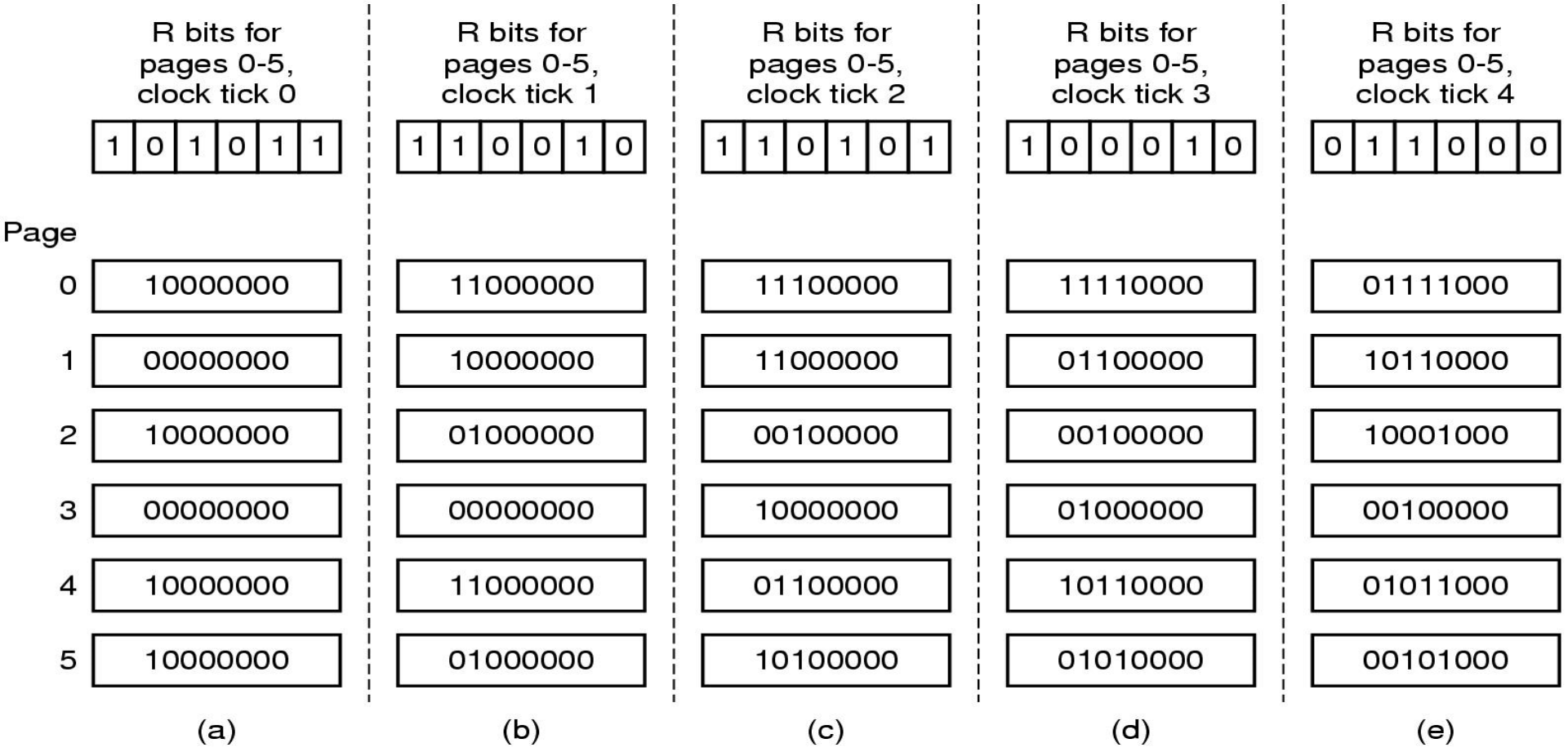


LRU using a matrix – pages referenced in order 0,1,2,3,2,1,0,3,2,3

Simulating LRU in Software

- LRU hardware is not usually available.
  - NRU: evict a page that is NOT recently used;
    LRU: evict a page that is LEAST recently used;
- NFU (Not Frequently Used) is implemented in software.
  - At each clock interrupt, the R bit is added to the counter associated with each page. When a page fault occurs, the page with the lowest counter is replaced.
  - Problem: NFU never forgets, so a page referenced frequency long ago may have the highest counter.
- Modified NFU = NFU with Aging - at each clock interrupt:
  - the counters are shifted right one bit, and
  - the R bits are added to the leftmost bit.
  - In this way, we can give higher priority to recent R values.

Simulating LRU in Software



- The aging algorithm simulates LRU in software
- Note 6 pages for 5 clock ticks, (a)~(e)

LRU

12 references,
10 faults

Page Refs	3 Page Frames			
	Fault?	Page Contents		
A	yes	A		
B	yes	B	A	
C	yes	C	B	A
D	yes	D	C	B
A	yes	A	D	C
B	yes	B	A	D
E	yes	E	B	A
A	no	A	E	B
B	no	B	A	E
C	yes	C	B	A
D	yes	D	C	B
E	yes	E	D	C

LRU and Anomalies

Anomalies cannot occur.

12 references, 8 faults

Page Refs	4 Page Frames				
	Fault?	Page Contents			
A	yes	A			
B	yes	B	A		
C	yes	C	B	A	
D	yes	D	C	B	A
A	no	A	D	C	B
B	no	B	A	D	C
E	yes	E	B	A	D
A	no	A	E	B	D
B	no	B	A	E	D
C	yes	C	B	A	E
D	yes	D	C	B	A
E	yes	E	D	C	B

The Working Set Page Replacement Algorithm (1)

- The working set is the set of pages used by the *k* most recent memory references
- w(k,t) is the size of the working set at time, *t*

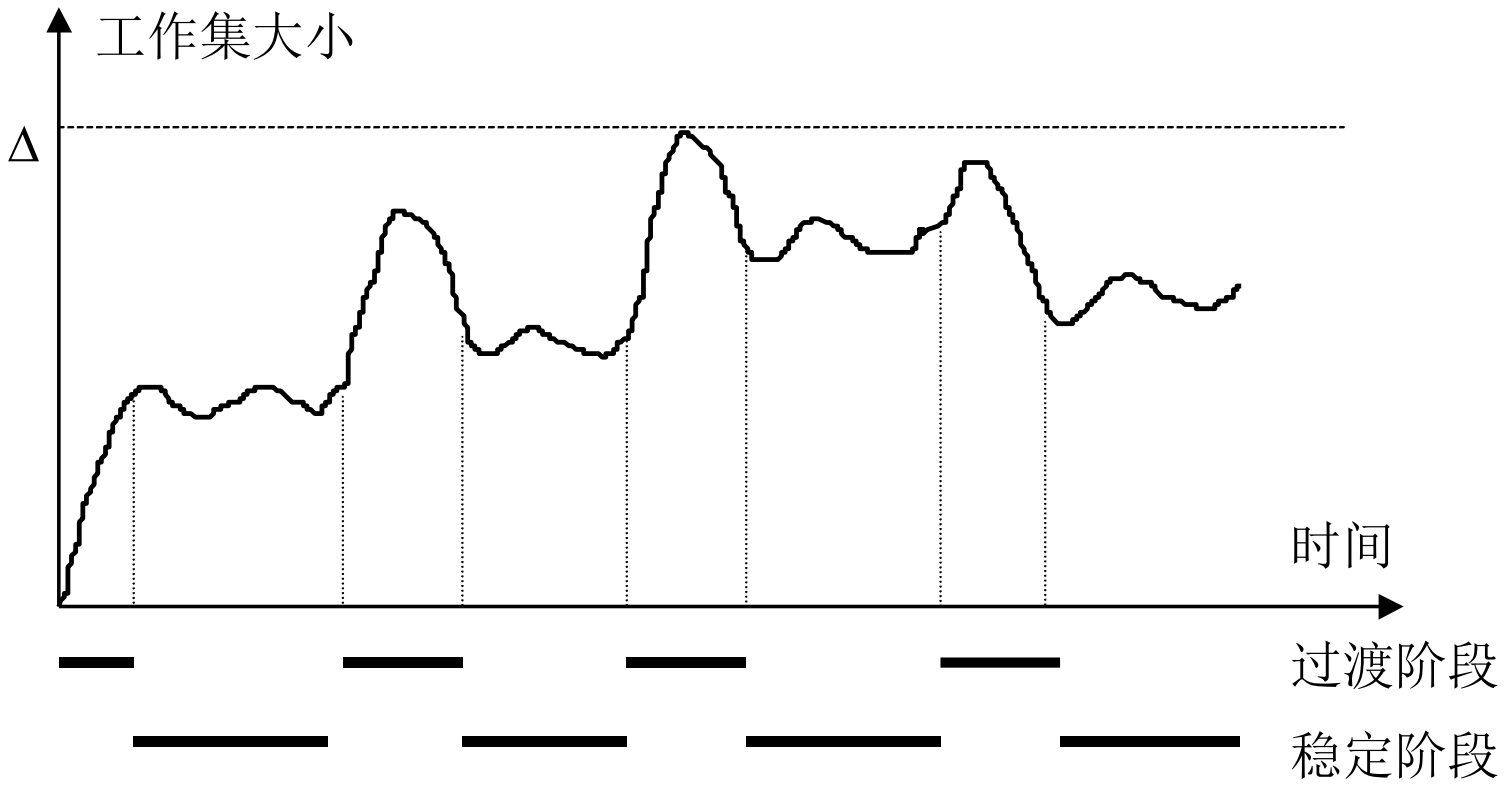
Example:

2615777516234123444343444 1327

| | t_1 | | t_2

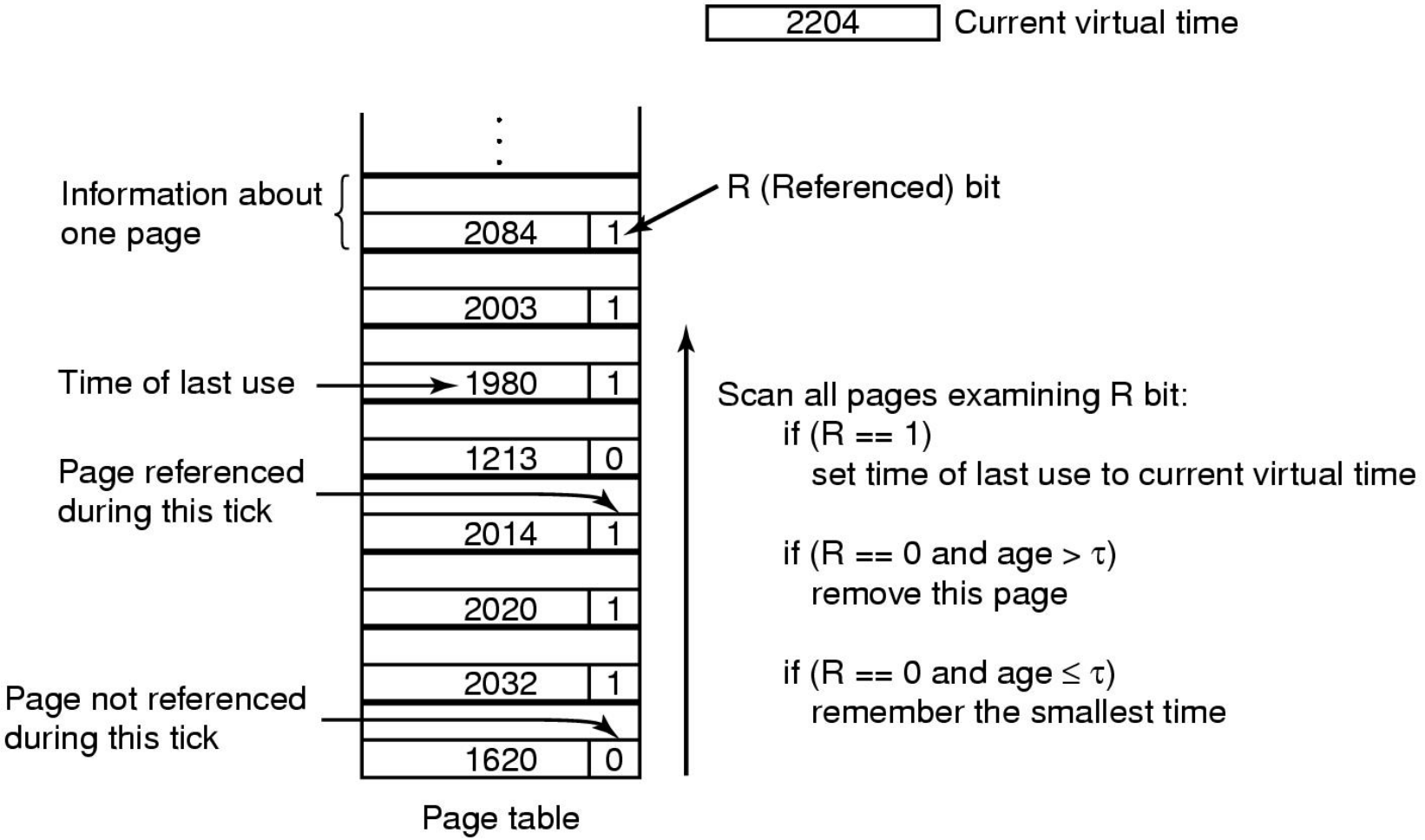
$ws(t_1)=\{1,2,5,6,7\}$

$ws(t_2)=\{3,4\}$



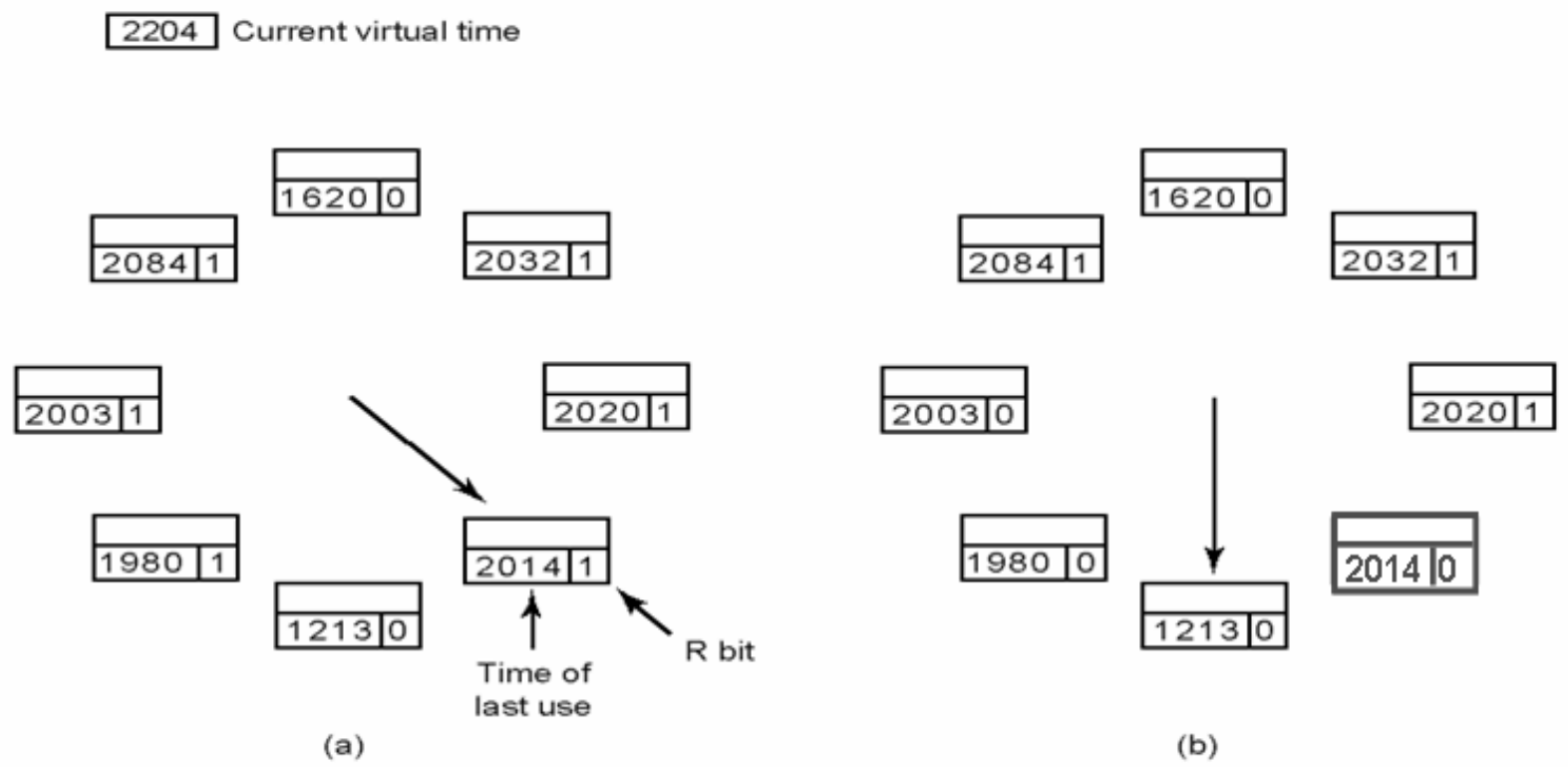
进程开始执行后，随着访问新页面逐步建立较稳定的工作集。
当内存访问的局部性区域的位置大致稳定时，工作集大小也大致稳定；
局部性区域的位置改变时，工作集快速扩张和收缩过渡到下一个稳定值。

The Working Set Page Replacement Algorithm (2)



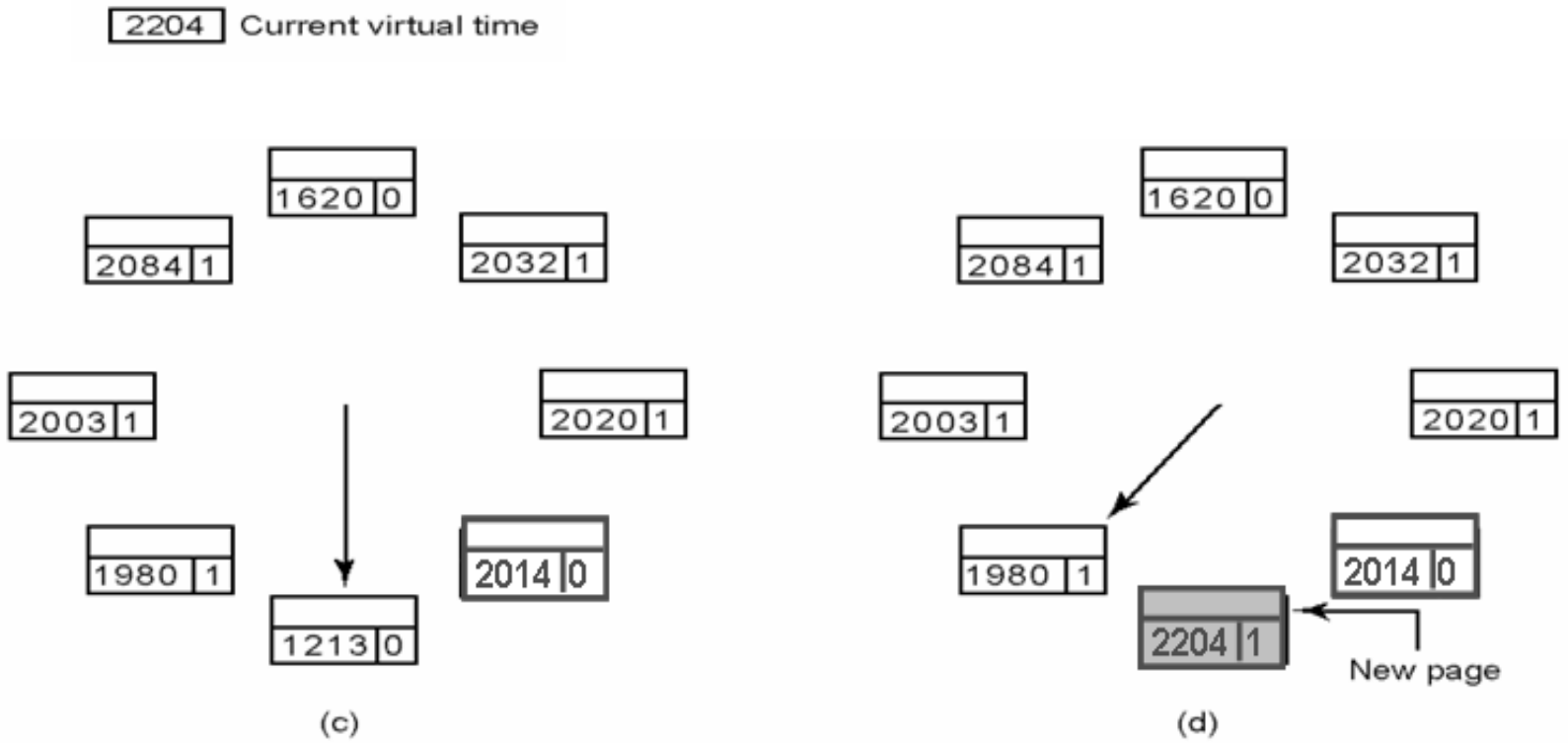
The working set algorithm

The WSClock Page Replacement Algorithm



Operation of the WSClock algorithm.
(a) and (b) give an example of what happens when R = 1.

The WSClock Page Replacement Algorithm



Operation of the WSClock algorithm.
(c) and (d) give an example of R = 0.

Review of Page Replacement Algorithms

Algorithm	Comment
Optimal	Not implementable, but useful as a benchmark
NRU (Not Recently Used)	Very crude
FIFO (First-In, First-Out)	Might throw out important pages
Second chance	Big improvement over FIFO
Clock	Realistic
LRU (Least Recently Used)	Excellent, but difficult to implement exactly
NFU (Not Frequently Used)	Fairly crude approximation to LRU
Aging	Efficient algorithm that approximates LRU well
Working set	Somewhat expensive to implement
WSClock	Good efficient algorithm